

## 1. funcionalidad:

- **F01-monitoreo:** El sistema debe medir el peso de los productos en los estantes mediante muestreo periódico (polling) utilizando celdas de carga y amplificadores HX711, procesando las variaciones de forma local en el microcontrolador.
- **F02-capacidad por microcontrolador:** Un único microcontrolador ESP32 debe ser capaz de monitorear hasta 5 estantes simultáneamente.
- **F03 - Comunicación bidireccional local:** Los nodos (ESP32) deben comunicarse con el Edge Gateway (Raspberry Pi) usando MQTT para reportar cambios y recibir parámetros.
- **F04 - Inventario Automático Constante:** El sistema debe detectar cambios físicos en tiempo real para mantener un conteo exacto del stock actual sin intervención humana.
- **F05 - Gestión de Stock Bajo (Umbrales):** El sistema debe permitir al usuario configurar umbrales personalizados (en porcentaje o kilogramos) para definir qué se considera "stock bajo" para cada producto.
- **F06 - Automatización de Pedidos y Alertas:** Al detectar stock bajo, el sistema actuará según la configuración:
  - *Modo Auto-stock:* Cargará automáticamente un pedido al proveedor para la fecha más pronta.
  - *Modo Manual:* Enviará una alerta/notificación inmediata a la aplicación del usuario.
- **F07 - Detección de "Robo Hormiga":** El sistema debe comparar la cantidad de producto que sale físicamente del estante contra lo registrado como "vendido". Cualquier discrepancia generará una alerta de posible robo.
- **F08 - Prevención de Sobre-stock y Aprendizaje Estacional:** Mediante algoritmos, el sistema debe aprender patrones de venta (ej. fechas donde se vende más o se desperdicia más un producto) para evitar compras excesivas y alertar sobre caducidades/mermas.
- **F09 - Sugerencias para Inventario Estancado:** El sistema debe detectar productos de lento movimiento y generar sugerencias de venta (ej. promociones, acomodo) extrayendo información de la red.
- **F10 - Reportes Automatizados de Salud del Negocio:** El sistema debe compilar y enviar reportes periódicos, ajustables a reporte semanal / mensual / trimestral / semestral o anual, con un resumen general del negocio: top de ventas, alertas de robo acumuladas, stock crítico y las sugerencias generadas.

## 2. Usabilidad

- **U01 - Dashboards y Analítica:** La aplicación debe mostrar gráficos intuitivos sobre el estado del negocio, histórico de ventas de los estantes y patrones detectados.
- **U02 - Interfaz "Digerible":** La información, especialmente las sugerencias de ventas y analíticas, debe presentarse en un lenguaje claro, sin tecnicismos, pensado para dueños de negocios que no son expertos en tecnología.
- **U03 - Monitoreo Multi-sucursal:** El sistema debe contar con un panel centralizado (Dashboard) que permita visualizar el estado de múltiples sucursales desde una sola cuenta.
- **U04 - Restricción de estante:** El sistema validará que se asigne un único tipo de producto por cada estante físico para dar sentido a las métricas.
- **U05 - Frontend Moderno:** La aplicación web/ móvil debe estar desarrollada en React native para garantizar fluidez y responsividad multiplataforma.
- **U.06 - Exportación de Reportes:** Los reportes de salud del negocio deben ser accesibles desde la aplicación y fácilmente exportables o entregables en formatos comunes (excel, PDF, notificaciones por correo electrónico).

## 3. confiabilidad:

- **R01 - Funcionamiento Offline:** El Edge Gateway (Raspberry Pi) debe seguir operando y recolectando datos en la red local MQTT sin internet.
- **R02 - Persistencia de datos local:** Ante caídas de red, los datos esenciales se guardarán temporalmente en una base de datos local (SQLite).
- **R03 - Patrón "Guarda y Envía":** Mecanismo de *store and forward* para asegurar la integridad de los datos durante las desconexiones.
- **R04 - Sincronización progresiva:** Al volver el internet, la Raspberry subirá los datos locales a la nube (PostgreSQL/InfluxDB) de forma progresiva.
- **R05 - Arquitectura Híbrida:** Garantía de alta disponibilidad combinando procesamiento local y almacenamiento en la nube.

## 4. Rendimiento

- **P01 - Eficiencia de transmisión MQTT:** El ESP32 solo debe transmitir datos al Gateway cuando detecte un cambio real superando un umbral, evitando saturar la red WiFi.
- **P02 - Edge Computing:** El filtrado de ruido de las celdas de carga se realizará localmente en la Raspberry Pi.
- **P03 - Delegación Analítica:** Los algoritmos de aprendizaje (estacionalidad, sobre-stock) se ejecutarán exclusivamente en el

backend de la nube (Python) para no saturar la Raspberry.

- **P04 - Gestión centralizada:** La arquitectura nube debe soportar peticiones concurrentes de múltiples estantes y sucursales.

## 5. Soportabilidad:

- **S01 - Adaptabilidad al Rubro:** El sistema (software) debe ser agnóstico al tipo de producto, permitiendo adaptarse a supermercados, ferreterías, farmacias, etc.
- **S02 - Contenerización:** El entorno de la Raspberry Pi usará contenedores (Docker) para aislar Mosquitto y SQLite.
- **S03 - Escalabilidad "Plug & Play":** Agregar nuevos estantes (ESP32) a la red debe ser rápido y sin reconfigurar la arquitectura base.
- **S04 - Modelo SaaS:** Backend diseñado para ofrecerse como Software as a Service, centralizando actualizaciones y mantenimiento.
- **S05 - Mantenibilidad del Firmware:** Código en C utilizando ESP-IDF para el microcontrolador.